

Bus Reservation System

What can be released as GitHub packages

Repository	Jashan-randhawa/Bus-Resrvation
Author	Jashan Randhawa
Analysed commit	e52b511 (main)
Latest tag	v2.2.0
Stack	PHP 8.4 / Apache / MySQL (TiDB Cloud), Docker, Render

1. Executive summary

The repository already publishes one GitHub package: a Docker image on GitHub Container Registry (GHCR) built by `.github/workflows/docker-publish.yml`. The remaining work is to correct how that image is tagged, add a downloadable release bundle, and then decide whether to extract reusable code as npm or Composer packages.

Key findings

- **Docker image is live but mis-tagged.** latest follows every push to main, and a hardcoded `v2.0.0` tag is overwritten on every build.
- **Release hygiene gap.** Tags `v1.0.0` to `v2.2.0` exist, but `CHANGELOG.md` has no `v2.2.0` section and its top entry is still *Unreleased*.
- **Seat picker is not yet package-ready.** `assets/js/seat-map.js` hardcodes element IDs, a relative `api-seats.php` URL and Bootstrap class names.
- **Composer extraction needs a refactor first.** Booking logic lives in global functions in `includes/helpers.php` (about 1,300 lines) coupled to `mysqli`, `sessions` and `mail`.
- **The earlier plan PDF is stale.** It describes `v2.0.0` and PHP 8.1; the repo is on PHP 8.4 and the image name is `bus-resrvation`.

2. Current state

Area	Finding	Evidence
Container package	Published to ghcr.io on push to main, on <code>v*.*.*</code> tags and on release	<code>docker-publish.yml</code>
Base image	<code>php:8.4-apache</code> , <code>mysqli</code> , Apache security config, healthcheck	Dockerfile
Git tags	<code>v1.0.0</code> , <code>v2.0.0</code> , <code>v2.1.0</code> , <code>v2.2.0</code>	<code>git ls-remote --tags</code>
Changelog	Sections: <i>Unreleased</i> , <code>2.1.0</code> and older; none for <code>2.2.0</code>	<code>CHANGELOG.md</code>
CI	PHP 8.3 and 8.4 lint, composer validate, SQL anti-pattern grep	<code>ci.yml</code>
Composer manifest	type project, no autoload, requires <code>php >=8.3</code> and <code>ext-mysqli</code>	<code>composer.json</code>
Tests	Plain PHP scripts run by <code>tests/run_tests.php</code> ; not part of CI	<code>tests/</code> , <code>ci.yml</code>
Docs	Wiki folder (7 pages) and a packages plan PDF	<code>wiki/</code> , <code>docs/</code>

3. Problems in the current publishing setup

P1	latest tracks main, not releases		High
Problem	The metadata step applies <code>type=raw, value=latest</code> unconditionally, so any commit pushed to main moves latest.		
Location	<code>.github/workflows/docker-publish.yml</code> , <code>docker/metadata-action</code> tags block.		
Impact	Anyone running <code>docker pull ...:latest</code> can receive unreleased or half-finished work, such as the current Unreleased changelog items.		
Solution	Enable latest only for tag builds and add an edge tag for main (corrected workflow in section 5).		
P2	Hardcoded v2.0.0 tag is overwritten		High
Problem	<code>type=raw, value=v2.0.0</code> is a literal. Every build re-points the v2.0.0 tag to whatever was just built.		
Location	Same file, same tags block.		
Impact	Version tags are meant to be immutable. Here v2.0.0 silently changes content, which breaks reproducible deployments and rollbacks.		
Solution	Delete the line. Let <code>type=semver</code> generate version tags from the git tag only.		
P3	Release and tag triggers can duplicate runs		Low
Problem	The workflow listens to both <code>push: tags</code> and <code>release: published</code> . Publishing a release that creates a new tag can start two builds of the same image.		
Location	<code>on:</code> block of <code>docker-publish.yml</code> .		
Impact	Wasted Actions minutes and a race to push the same tags.		
Solution	Keep the tag trigger and <code>workflow_dispatch</code> ; drop the release trigger (the release bundle in section 6 can use it instead).		
P4	Tag v2.2.0 has no changelog entry		Medium
Problem	<code>CHANGELOG.md</code> shows Unreleased on top, then 2.1.0. The v2.2.0 tag exists on the remote with no matching section.		
Location	<code>CHANGELOG.md</code> , lines near the top.		
Impact	Release notes and package consumers cannot tell what changed in 2.2.0.		
Solution	Move the Unreleased items under <code>## [2.2.0] - date</code> and start a fresh Unreleased section.		
P5	Packages plan PDF is outdated		Low
Problem	The existing PDF says v2.0.0, PHP 8.1 and image name <code>bus-reservation</code> , and proposes a workflow that already exists.		
Location	<code>docs/Bus_Reservation_GitHub_Packages_Plan.pdf</code> .		
Impact	Misleads contributors and contradicts the real image name.		
Solution	Replace it with this document.		

4. Package candidates

Candidate	Registry	Readiness	Effort	Verdict
Docker image	ghcr.io	Published; tagging needs fixes	Low	Fix now
Release bundle (zip)	GitHub Releases	Not started	Low	Do next
Seat picker widget	npm (GitHub Packages or npmjs)	Needs refactor: hardcoded IDs, endpoint, Bootstrap classes	Medium	Optional
Core booking library	Packagist (Composer)	Needs refactor: global functions coupled to mysqli, session, mail	High	Later

A note on registries: GitHub Packages supports container images, npm and others, but it has no PHP Composer registry. Composer packages are distributed through Packagist, which reads the GitHub repository directly.

4.1 Seat picker (npm) - what blocks extraction

- `seat-map.js` (181 lines) looks up fixed IDs: `seat-grid`, `submit-booking-btn`, `summary-seat-display`, `seat-live`, `booking-form`.
- It polls `api-seats.php` with a relative URL and `credentials: same-origin`; a package needs this endpoint to be an option.
- It builds Bootstrap badge classes (`badge badge-success`) in the summary line.
- It also owns form-submit handling and `bfcache` (pageshow) recovery, which are app behaviour, not widget behaviour.
- The markup is generated by PHP in `includes/seat-map.php` and styled by `assets/css/seat-map.css` (673 lines), so a package would ship the JS and CSS together with a documented HTML contract.

4.2 Core library (Composer) - what blocks extraction

- `includes/helpers.php` mixes HTML escaping, CSRF, flash messages, throttling, seat logic, booking transactions, mail and CSV export in one file of global functions.
- `create_booking()` (about 230 lines) depends on the app schema, `seat_lock` table and application timezone constants.
- Most self-contained candidates: `build_seat_layout()`, `validate_person_fields()`, `mask_phone()`, `mask_email()`, and the PNR token generation.
- `composer.json` would need type: `library`, a PSR-4 autoload section and a `src/` directory; the current file is an application manifest.

5. Corrected Docker workflow

Replace the `on:` and `metadata` sections of `docker-publish.yml` as below. Everything else (`login`, `buildx`, `cache`, `build-push`) stays.

```
on:
  push:
    branches: [ "main" ]
    tags: [ 'v*.*.*' ]
  workflow_dispatch:

# ...unchanged: env, permissions, checkout, buildx, login, lowercase step...

- name: Extract Docker metadata (tags, labels)
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: ${ env.REGISTRY }/${ steps.image_name.outputs.IMAGE }
    tags: |
      type=semver,pattern={{version}}
      type=semver,pattern={{major}}.{{minor}}
      type=raw,value=latest,enable=${ startsWith(github.ref, 'refs/tags/v') }
      type=edge,branch=main
```

Result: pushing tag `v2.2.0` produces `2.2.0`, `2.2` and `latest`; pushing to `main` produces only `edge`. Version tags are never reassigned.

One-time GitHub setting: open the package page for the image and set visibility to `Public`, and link it to the repository so the `Packages` panel appears on the repo home page.

6. Release bundle workflow

GitHub's automatic source archives do not include a curated layout. This workflow attaches a clean zip to every published release, containing source, `database/` (`init.sql` and migrations) and `.env.example`, and leaving out git metadata, CI files, docs and unused images.

```
# .github/workflows/release-bundle.yml
name: Release bundle
on:
  release:
    types: [published]
permissions:
  contents: write
jobs:
  bundle:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build archive
        run: |
          zip -r "$RUNNER_TEMP/bus-reservation-${GITHUB_REF_NAME}.zip" . \
            -x ".git/*" ".github/*" "docs/*" "assets/images/_unused/*"
      - name: Upload to release
        env:
          GH_TOKEN: ${ secrets.GITHUB_TOKEN }
        run: gh release upload "$GITHUB_REF_NAME" \
          "$RUNNER_TEMP/bus-reservation-${GITHUB_REF_NAME}.zip" --clobber
```

7. Roadmap

Phase	Work	Files touched	Done when
1. Fix the image	Apply corrected tags block; drop release trigger	docker-publish.yml	Main push yields only edge; tag push yields version and latest
2. Cut a clean release	Add 2.2.0 section to changelog; create GitHub Release for v2.2.0	CHANGELOG.md	Release page shows notes; package page shows 2.2.0
3. Release bundle	Add release-bundle workflow	release-bundle.yml	Zip appears under release assets
4. Docs and badges	Add pull command and Packages badge to README; delete old plan PDF	README.md, docs/	README documents docker pull and run with env vars
5. Seat picker (optional)	Parameterise selectors and endpoint, remove Bootstrap classes, add README and package.json, publish to npm	assets/js/seat-map.js, new package folder	App still works using the packaged build
6. Core library (later)	Introduce src/ with PSR-4 classes, move pure helpers first, add tests to CI, register on Packagist	includes/helpers.php, composer.json, ci.yml	App requires its own library via Composer

8. Verification checklist

- After phase 1, push a trivial commit to main and confirm only the edge tag changes in the package's tag list.
- Push a test tag (for example `v2.2.1-rc.1`) and confirm no `latest` tag is created for a prerelease.
- Run `docker pull ghcr.io/jashan-randhawa/bus-resrvation:2.2.0` from a logged-out shell to confirm the package is public.
- Start the container with the database environment variables from `.env.example` and confirm the healthcheck passes.
- Download the release zip and confirm it contains `database/init.sql` and `.env.example`, and no `.git` folder.

Scope note: this plan is based on a read-only clone of the main branch. No repository files were changed.